# AgentCoder

## Multi-Agent Code Generation with Iterative Testing and Optimisation

*Huang et al. — arXiv:2312.13010*

Presented by: Alon Engel, Anas Khoury, Othman Khleel

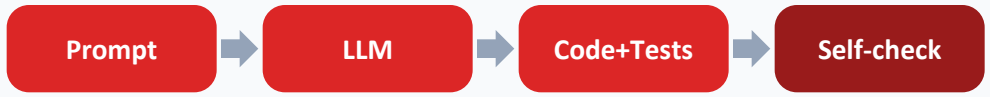
# The Problem

*When the same model writes code AND tests, it cheats on itself*

## Single-Agent Approach

- Same LLM writes code and tests
- Tests inherit bugs from the code
- No objectivity — like a student grading their own exam
- Edge cases get missed

**Developer flow:**




## Existing Multi-Agent Frameworks

- MetaGPT: 5 agents,  ChatDev: 7 agents
- High token cost from agent coordination
- Test accuracy only ~80%
- Tests not based on real execution

**Developer flow:**

# The Solution: 3 Specialized Agents

*Each agent has one job — and does it well*

## Programmer Agent 1

כותב את הקוד

**LLM**

Uses Chain-of-Thought:
- Understand task
- Choose algorithm
- Write pseudocode
- Generate code

## Test Designer Agent 2

מעצב בדיקות

**LLM**

Independent of code:
- Basic test cases
- Edge cases
- Large-scale tests
- No bias!

## Test Executor Agent 3

מריץ ובודק

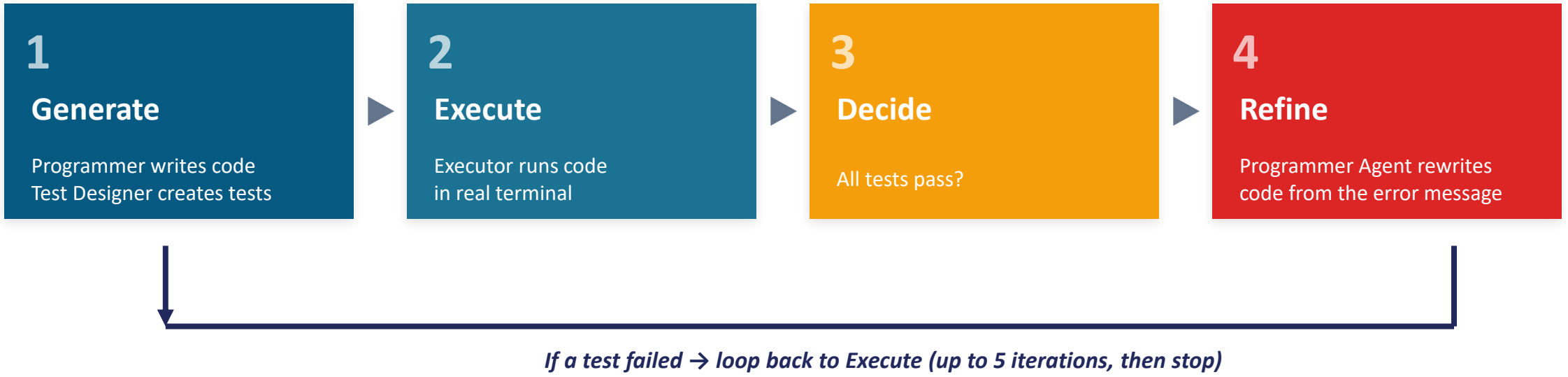
**Script**

Python script (**not LLM!**):
- Runs code in terminal
- Captures real errors
- Sends feedback
- **0 tokens cost!**

***Key Insight: Test Designer never sees the code → objective tests, no inherited bias***

# The Iterative Feedback Loop

*The heart of AgentCoder — like TDD (Test Driven Development) for AI*




### Why is this powerful?

- Concrete feedback — actual errors from terminal, not LLM guesses
- Programmer Agent fixes itself — LLM rewrites the code from the error; no human in the loop
- Tests stay fixed — Programmer Agent can't 'convince' the Test Designer to change its mind
- **Bounded budget —** paper caps the loop at 5 iterations; most fixes happen in the first 1–2 (74% → 80% over 5 rounds)

# The Original AgentCoder Pipeline

*Figure 1 — the full pipeline with a HumanEval code-generation example*

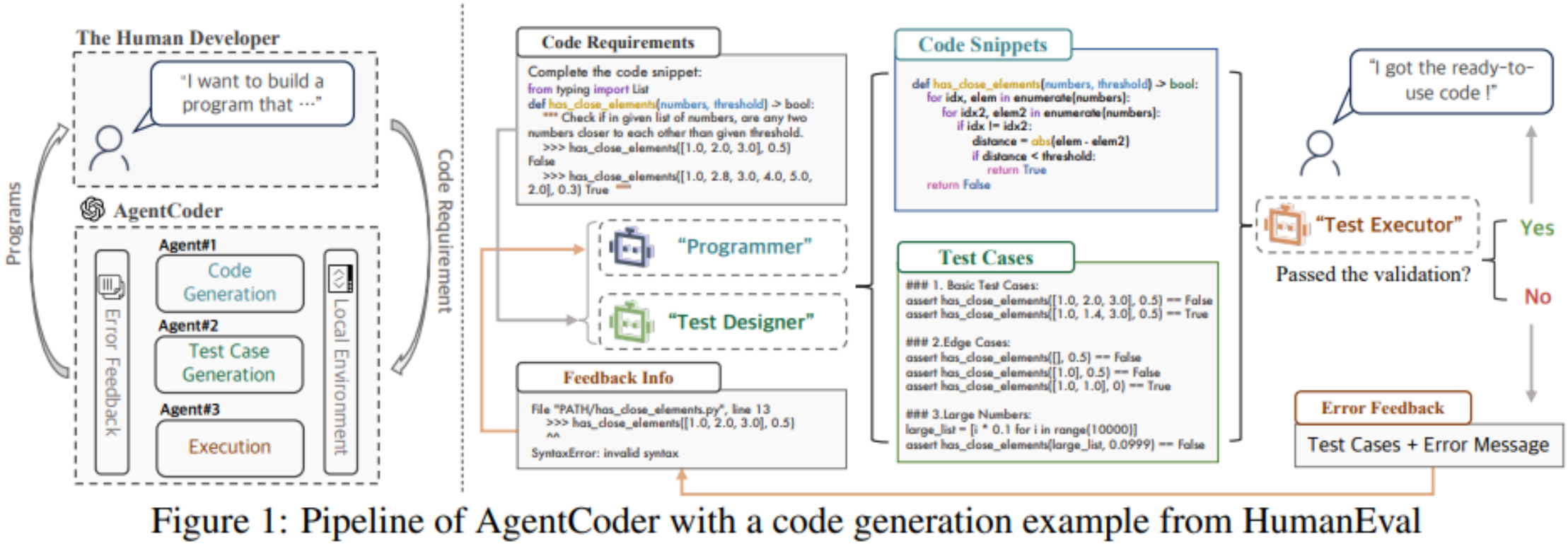



Figure 1: Pipeline of AgentCoder with a code generation example from HumanEval

Key idea: tests are written independently of the code, so the executor's feedback is objective — driving each refinement loop.

*Source: Huang et al. (2023), "AgentCoder," arXiv:2312.13010*

# Task Demonstration: has_close_elements

*Check if any two numbers in a list are closer than a given threshold*

📝 **Programmer's first attempt**

```
if numbers[i] - numbers[j] <= threshold:
    return True
```

❌ *Bug: missing abs() — fails on negative differences*

🧪 **Test Designer (independent python code!)**

```
# Basic
assert has_close_elements([1.0, 2.5], 1.0) == True
# Edge case
assert has_close_elements([], 1.0) == False
# Large scale
list_100k = [i * 0.1 for i in range(100000)]
```

⚙️ **Test Executor runs the code → AssertionError on negative numbers**

Concrete error message sent back to Programmer (not a guess — actual runtime output)

→ **Programmer fixes: abs(numbers[i] - numbers[j]) ≤ threshold** ✓

**Why this works:**

Single-agent approaches (like CodeCoT) wrote both the buggy code AND biased tests that didn't catch the bug. AgentCoder's independent test designer caught it because it was based on the requirement, not the code.

# Results: Better AND Cheaper

*Counter-intuitive finding: more agents does NOT mean better results*

**96.3%**
**91.8%**

**pass@1**

HumanEval — vs 90.2% state-of-the-art
MBPP — vs 78.9% state-of-the-art

**57K**
**66K**

**tokens vs MetaGPT**

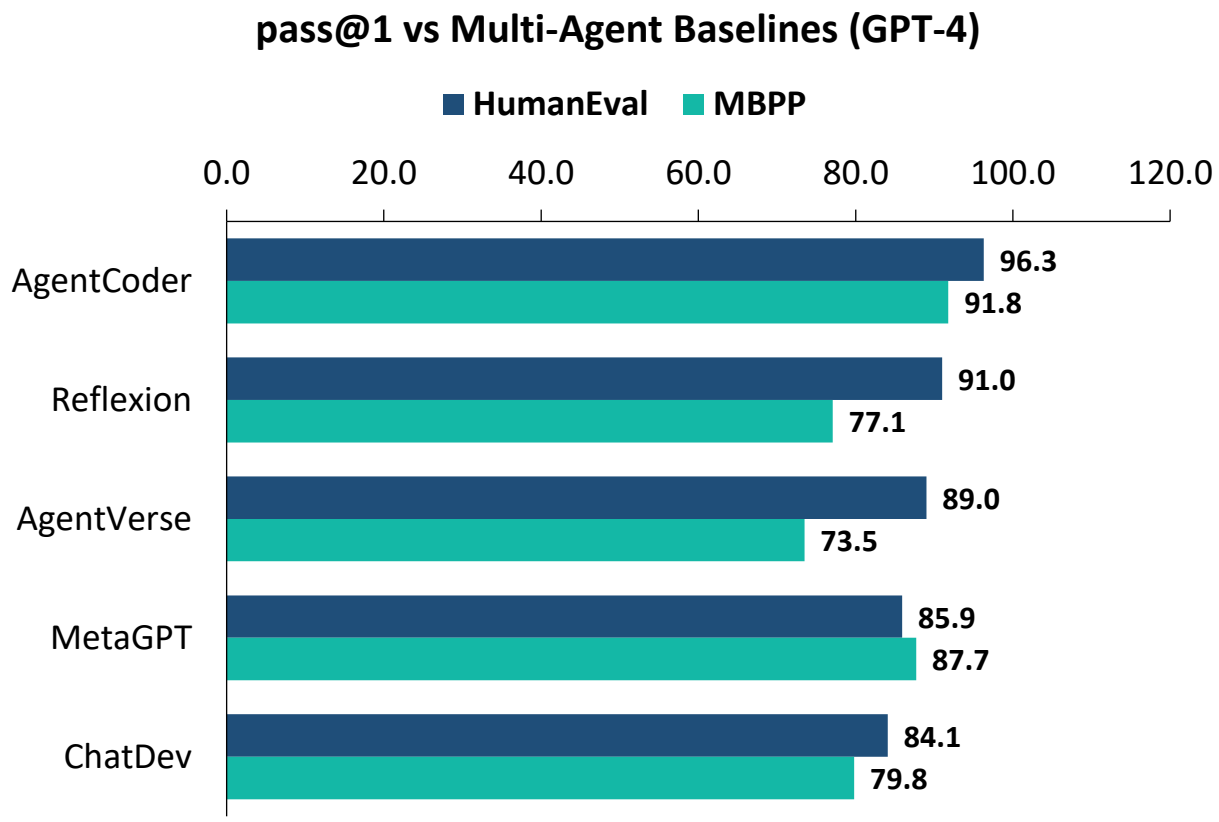
HumanEval — vs 138K (~59% less)
MBPP — vs 207K (~68% less)

**3**

**agents only (vs 7 in ChatDev)**

*less coordination overhead*

**Accuracy across both benchmarks**




🎯 **Beats every multi-agent baseline on both HumanEval AND MBPP**

💰 **Up to ~70% fewer tokens than other multi-agent frameworks — better AND cheaper**

# Why Separate the Agents?

*RQ6 — specialized agents beat one agent doing both, on every metric (shown as HumanEval / MBPP)*

| Metric | Single Agent | AgentCoder (Multi-Agent) |
|---|---|---|
| **Test-case accuracy** | 61.0 / 51.8 | **87.8 / 89.9** |
| **pass@1 (code)** | 71.3 / 79.4 | **79.9 / 89.9** |
| **Test line coverage** | 72.5 / 75.9 | **87.5 / 89.5** |

Why it works: a separate test designer can't be biased by code it never wrote, and each agent stays focused on a single job — so the tests are tougher, more objective, and catch the edge cases a single agent would rationalize away.

*Source: Huang et al. (2023), "AgentCoder," arXiv:2312.13010 (RQ6, Tables 6–8)*

# Token Overhead — AgentCoder vs Multi-Agent Frameworks

*Total token cost per problem (GPT-4) — lower is better*

| Framework | HumanEval tokens | MBPP tokens |
|---|---|---|
| **AgentCoder** | **56.9K** | **66.3K** |
| MetaGPT | 138.2K | 206.5K |
| AgentVerse | 149.2K | 193.6K |
| ChatDev | 183.7K | 259.3K |

**HumanEval**: function completion · **MBPP**: entry-level Python tasks

💡 **2–4× fewer tokens — at higher accuracy**

*Source: AgentCoder paper (Huang et al., 2024) — Table 9, GPT-4*

# Extended Context: Agent Architecture

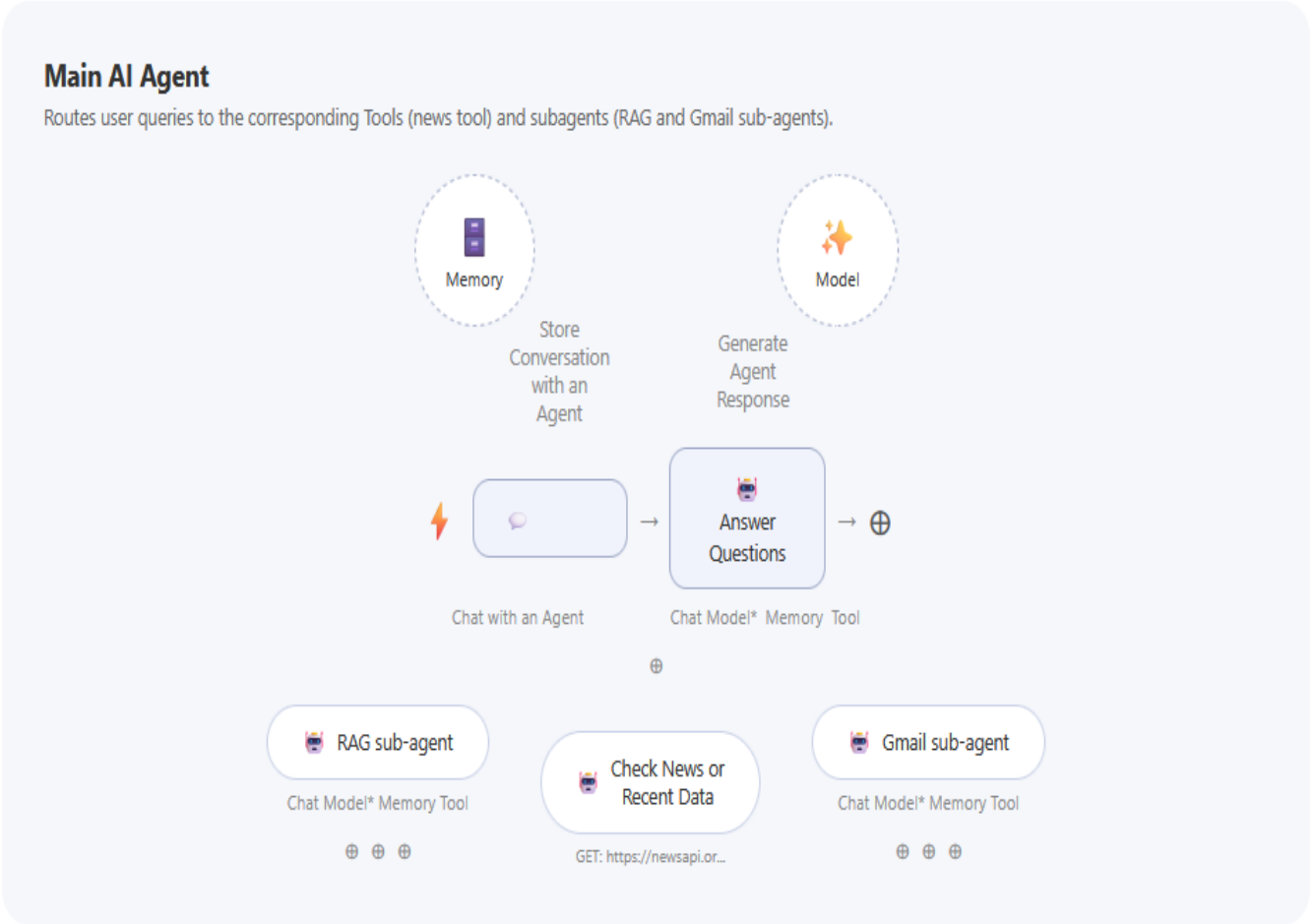
## Agentic Workflow

### 🔁 Iterative Loop:

The system does not stop at the first answer, but tests, executes, and improves in real time.

### 🧠 High-Order Thinking:

Moves beyond simple pattern matching (Pattern Matching) to lead toward a complex and structured solution (System 2).



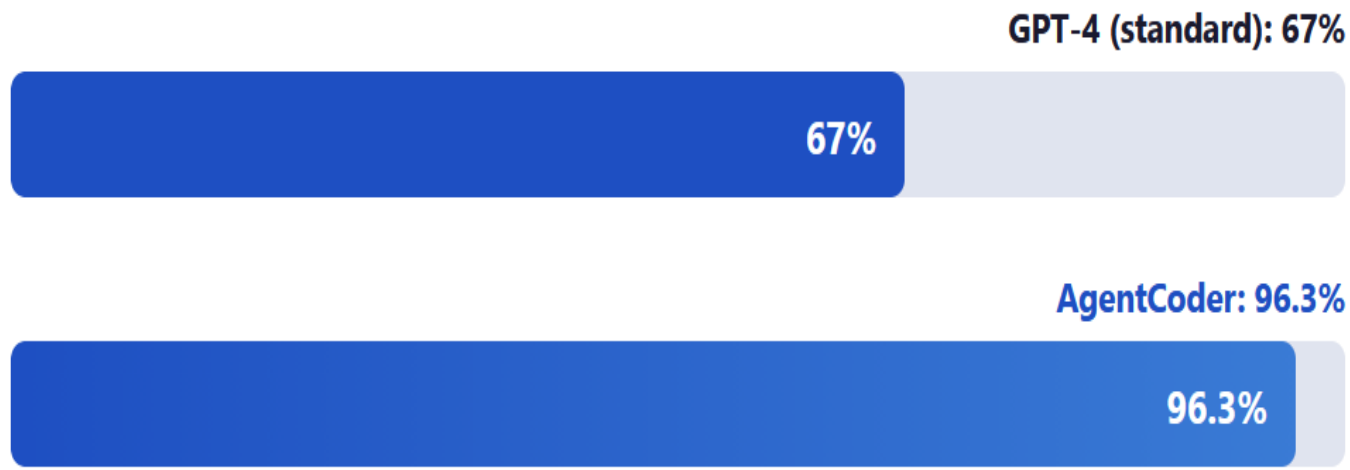
# Breakthrough Performance

**50%**

**Fewer Token costs than previous agent solutions**

*Using only 3 LLM calls*

**Accuracy on *HumanEval (Pass@1)***



# The Future: Software Engineering 2.0

From Copilot
to Autopilot

## Why does this work?

**Separation of Concerns:** The code reviewer sees only the code, not the requirements. ✅

**External Validation:** Uses a real Compiler for verification, not an LLM. ⚙️

**Hallucination Reduction:** Checks are based on the model's logical outputs. 🛡️

# Critical Lens: Where AgentCoder Falls Short

*Strong on toy benchmarks — but the design has real blind spots*

### Objective ≠ correct

The test designer is the same kind of LLM reading the same ambiguous prompt. Misread the spec and it writes wrong tests — then the executor's "objective" feedback steers the code confidently the wrong way.

### Proven only on toy problems

HumanEval & MBPP are short, single-function Python tasks likely seen in pre-training. 96.3% says little about multi-file, stateful, or under-specified real-world code.

### Self-written tests aren't ground truth

Production has no hidden tests. The loop only guarantees "passes the tests it wrote itself," which can miss whole edge cases — high pass@1 ≠ correctness.

### Narrow comfort zone

Needs a clear, auto-testable spec with deterministic I/O. Weak fit for UI, ML, or performance tasks — and iterating costs more tokens and latency than a single call.

*Critical analysis — based on Huang et al. (2023), arXiv:2312.13010, and the code-benchmark contamination literature.*

# The Field Moved On: Dec 2023 → 2026

*AgentCoder's core idea survived — its benchmark didn't*

### HumanEval is solved

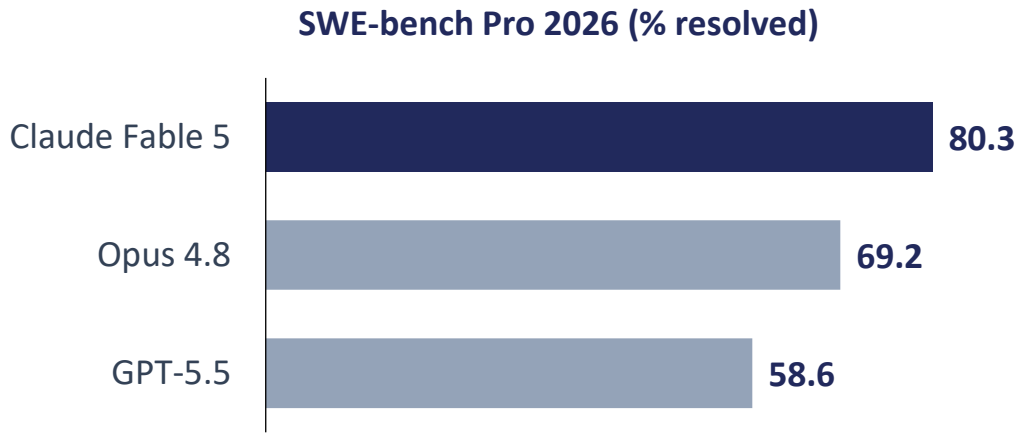
AgentCoder's 96.3% (2023) was already near the ceiling — today's best models sit around ~99%, so HumanEval no longer separates methods. The real bar moved to far harder, repo-level benchmarks.

### From functions to repositories

Evaluation shifted to real GitHub issues: SWE-bench and its Verified, Multilingual & Pro variants. Agents like SWE-agent and OpenHands edit whole codebases — a regime AgentCoder wasn't built for.

### The idea endures

Role specialization + execution feedback became standard (AlphaCodium, AgileCoder, CodeAct). Even 2026 frontier models do best when they get feedback as they work — AgentCoder's thesis.




*Through 2026 — HumanEval saturated; SWE-bench Pro the new bar; Claude Fable 5 / Mythos (Jun 2026).*

# Key Takeaways

*3 things to remember about AgentCoder*

**1**

## Separation of agents

Each agent focuses on one task. Tests stay objective because the test designer never sees the code.

*Like not letting a student grade their own exam*

**2**

## Iterative feedback loop

If tests fail → real error message → programmer refines → re-run tests → repeat until success.

*Like Test-Driven Development (TDD) for AI*

**3**

## Up to ~70% fewer tokens, higher accuracy

3 focused agents beat 5-7 chatty ones. Test Executor is a Python script — costs 0 tokens.

*Smart architecture > brute force*

***Smart architecture beat brute force in 2023 — but HumanEval is now solved; the real test has moved to repo-level SWE-bench.***

# Scan to Play: AgentCoder Escape Room